

■ Design Movie Ticket Booking (BookMyShow)

System Design Interview | E-Commerce & Marketplace

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional

Browse movies/shows; View seat map; Select & hold seats; Pay and confirm; E-ticket generation; Cancellation & refunds

Non-Functional

No double booking (strong consistency for seats); Handle release-day traffic spikes; <500ms seat lock; 99.9% availability

Scale

India: 20M tickets/month; Popular shows sell out in minutes; Seat selection is the critical concurrency challenge

■ High-Level Architecture

- Show Service (movies, theaters, schedules), Seat Service (inventory + locking), Booking Service (orchestration), Payment Service
- Seat map: stored in DB as matrix; each seat has status: AVAILABLE / HELD / BOOKED; real-time updates via WebSocket to all viewers
- CDN for movie posters, trailers; Elasticsearch for movie/show search with filters (city, language, date, genre)
- Separate read replicas for browsing (seat availability display) vs primary for seat locking (strong consistency needed)

■ Seat Locking — The Core Problem

- **Optimistic Locking:** Read seat status + version; UPDATE seat SET status=HELD, version=v+1 WHERE seat_id=X AND version=v AND status=AVAILABLE; retry if fails
- **Pessimistic Locking:** SELECT seat_id FROM seats WHERE show_id=X AND seat_id IN (...) FOR UPDATE; lock rows for duration of transaction; simpler but slower
- **Redis TTL Lock:** SETNX seat:{show_id}:{seat_id} user_id EX 600 — lock for 10 minutes; fast; eventual DB sync on payment success
- **Chosen Approach:** Redis SETNX for fast temporary hold (10 min TTL) + DB transaction on payment to make permanent; best of both worlds

■ Booking Flow

- Step 1: User selects seats → POST /hold-seats → Redis SETNX for each seat → success returns hold_token (valid 10 min)
- Step 2: User pays → Payment Service → on success: DB transaction marks seats BOOKED, removes Redis locks
- Step 3: E-ticket generated (QR code with booking_id) → sent via email/SMS; stored in S3
- Step 4: If payment fails or times out → Redis TTL releases hold → seats available again for others

- Concurrent users: if two users try same seat simultaneously → SETNX is atomic; only one wins; loser gets 'seat taken' immediately

■ Data Model

- shows: show_id, movie_id, theater_id, screen_id, start_time, total_seats, available_seats — PostgreSQL
- seats: seat_id, show_id, row, number, category(GOLD/SILVER/PLATINUM), status, booked_by_user_id — PostgreSQL
- bookings: booking_id, user_id, show_id, seat_ids[], total_amount, status, payment_id, created_at
- Index: (show_id, status) on seats for fast available seat lookup; (user_id) on bookings for history

■■ Seat Lock Strategy

DB Pessimistic Lock	Redis + DB Hybrid
✓ Guaranteed consistency	✓ Sub-millisecond lock check
✓ Simpler rollback	✓ Scales to millions
✓ Slower (DB I/O)	✓ TTL auto-releases
✓ Lock contention at scale	✓ Need Redis HA (Redlock)

■ Handling Release-Day Traffic

- Pre-sale queue: virtual waiting room 30 min before tickets open; queue assigns fair position
- Read/write split: seat availability display uses Redis cached count (eventual); actual lock uses primary DB
- Horizontal scaling: Seat Service is stateless (state in Redis/DB); add nodes behind load balancer for spike
- Circuit breaker: if Payment Service slow → queue payments; don't hold seats indefinitely; fail fast with retry